

PyCon Busan 2026 · 25 min talk

모르니까 만들어봤습니다. 웹 브라우저

— 파이썬으로 토이 브라우저를 만들다가, 도중에 탈주한 이야기

Jaeyeol Lee · @kodingwarrior · [kojima-yakshaving/browser-study](https://github.com/kojima-yakshaving/browser-study)

\$ whoami

이재열 (Jaeyeol Lee)

@kodingwarrior

- Neovim과 함께 운명공동체가 된 지 올해로 4년차
- INTP/ENFP를 왔다갔다하는 하이브리드형 인재
- LeetCode 같은 프로그래밍 퍼즐을 즐겨하는 편
- 커뮤니티를 여러곳 운영하면서 사회실험을 하는 사람
- yakshaving을 굉장히 좋아하는 사람



기존에 이미 좋은 기성품이 있음에도 불구하고,
동일한 동작을 하는 소프트웨어를 바닥부터 만드는
행위

밑바닥에서부터 만드는 웹브라우저
밑바닥에서부터 만드는 컴파일러
밑바닥에서부터 만드는 인터프리터

이 발표는요

브라우저를 '완전히 이해한' 이야기가 아닙니다.

끝까지 다 만들지도 못했습니다. 중간에 '탈주'했거든요.

'하드코어 모드' 스터디에서 먼저 달렸던 사람의 기록입니다.

연습문제 풀이 + 해설까지 쓰던 사람이었는데, 왜 도망갔을까요?

듣고 나면,

여러분도 중간에 끊겨도 괜찮으니깐, 일단 하드코어한 스터디를 하나쯤 해보고 싶어지길.

오늘 할 이야기

- 01  다시 끼게 된 스터디
2022년 경험자로 재참여, 하드코어한 버전으로
- 02  브라우저는 어떻게 동작하나요?
주소창에 g 키를 누르는 순간부터, 픽셀이 그려지기까지
- 03  책에서 벗어난 과감한 행동
책에서 알려주지 않는 것들이 있었기에 여러가지 면에서 색다른 시도를 했습니다
- 04  탈주 — 그리고 남은 것
먼저 풀던 사람이 왜 도망갔는가
- 05  AI가 다 짜주는 시대에, 왜?
밑바닥부터 짜는 훈련이 왜 여전히 필요한가
- 06  그래서 원하는 것
끝겨도 남는 스터디의 가치

잠깐, 그 전에 이야기가 좀 있습니다

사실 이 책으로 스터디를 한 게 이번이 처음이 아닙니다.

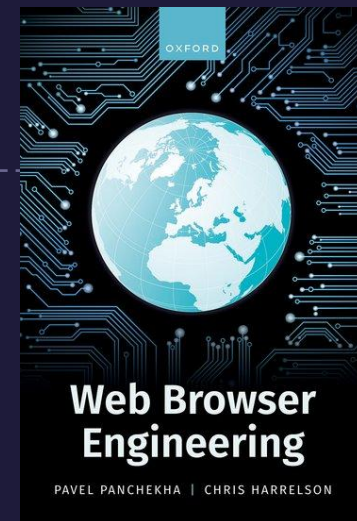
2022

SNS에서 모인 프론트엔드 개발자분들과, 처음으로 이 책을 펴습니다.

- 그땐 영어 원서가 최신이었고, 다른 챕터는 아직 집필 중이었습니다.
- 9챕터까지 진도를 나갔습니다. (지금 제가 멈춘 지점보다 더 갔던 셈입니다.)
- 그 후 스터디원 중 몇 분이 "면접에서 브라우저 동작을 잘 설명했다"는 간증을 남겼습니다.

그 후로,

- 2025년에 한국어 번역본이 나왔습니다.
- "언젠가 다시 해야지" 하고 버르고 있었습니다.
- ...그러다 누가 꼬드기더군요.

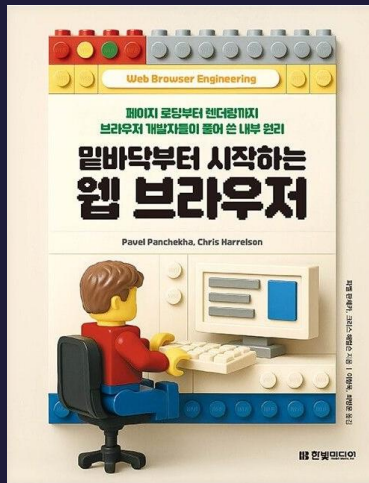


PART 1

다시 끼게 된 스터디

"누군가 코드기면 들어가야지" 하다가, 정말 코드겨져서

어쩌다 (또) 하게 됐다



2022

책 번역

2025.10

~ 탈주 이전까지

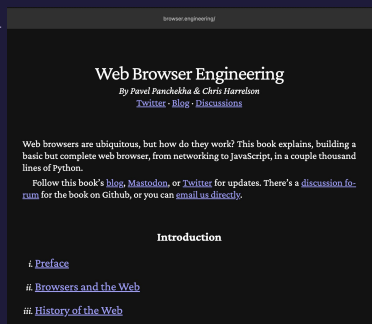
처음 스터디

국내 번역 출간

꼬드김 성공

하드코어 모드 참전

프론트엔드하는 지인들과
Web Browser Engineering



"다시 해야지" 마음만 먹음

어떤 팬클럽 디스코드에서
"같이 하실래요?"

연습문제 다 풀고
해설까지 쓰는 버전

핵심: 이번엔 '초심자'가 아니라 '두 번째'였습니다. 뽀뽀하게 해보고 싶었어요.

하드코어 모드 스터디의 규칙

1

연습문제를 '다' 푼다

Web Browser Engineering 교재는 챕터마다 연습문제가 5~9개씩 붙어 있음.
일단 전부 푼다.

2

풀이를 글로 남긴다

왜 그렇게 풀었는지, 어디서 막혔는지까지.
해커스펍 같은 블로그에 기록.

3

환경은 LLM에 맡긴다

단위테스트, CI, 실험용 서버는 *Claude Code* / *Codex*.
핵심 구현은 직접 손으로.

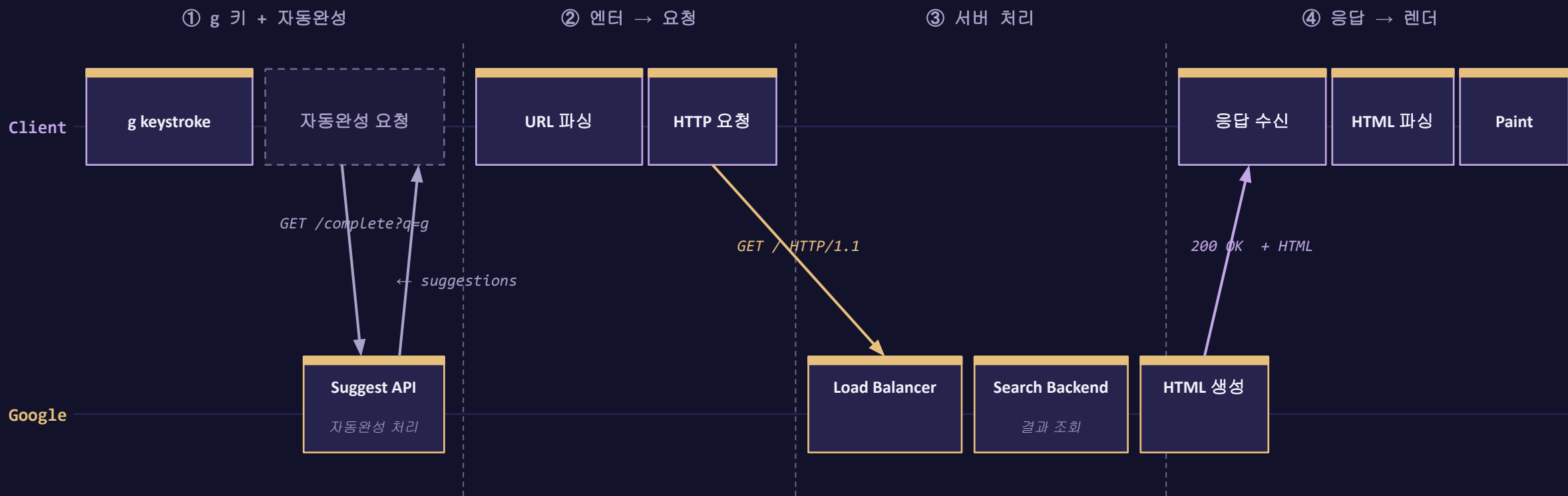
PART 2

브라우저는 어떻게 동작하나요?

주소창에 *g* 키를 누르는 순간부터, 픽셀이 그려지기까지

브라우저는 어떻게 동작하나요? (1/2)

“면접관 “주소창에 **g** 키를 눌렀을 때 일어나는 일을 설명해보세요.”



g 키 한 번에도 이미 수십 번 네트워크가 오갑니다. 엔터 후 HTML이 돌아오는 Client 쪽 과정을 손으로 짜봤어요.

브라우저는 어떻게 동작하나요? (2/2)

응답 HTML이 어떻게 '트리'가 되고, 그 트리가 어떻게 '그려지는가'.

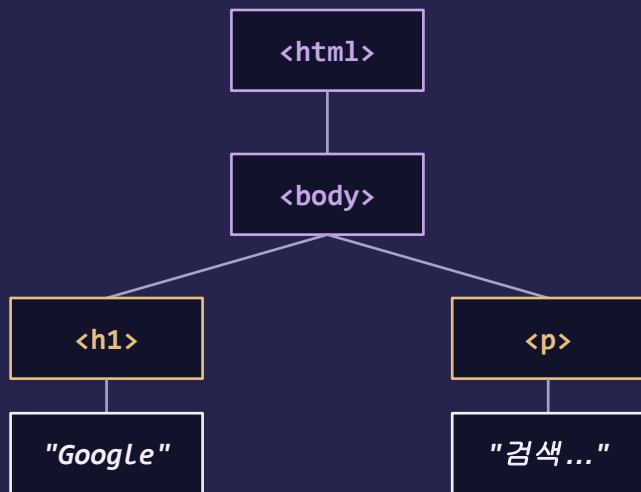
HTML (원문)

```
<html>
  <body>
    <h1>Google</h1>
    <p>검색 결과...</p>
  </body>
</html>
```

파싱



DOM 트리



그리기



화면 (Canvas)

google.com

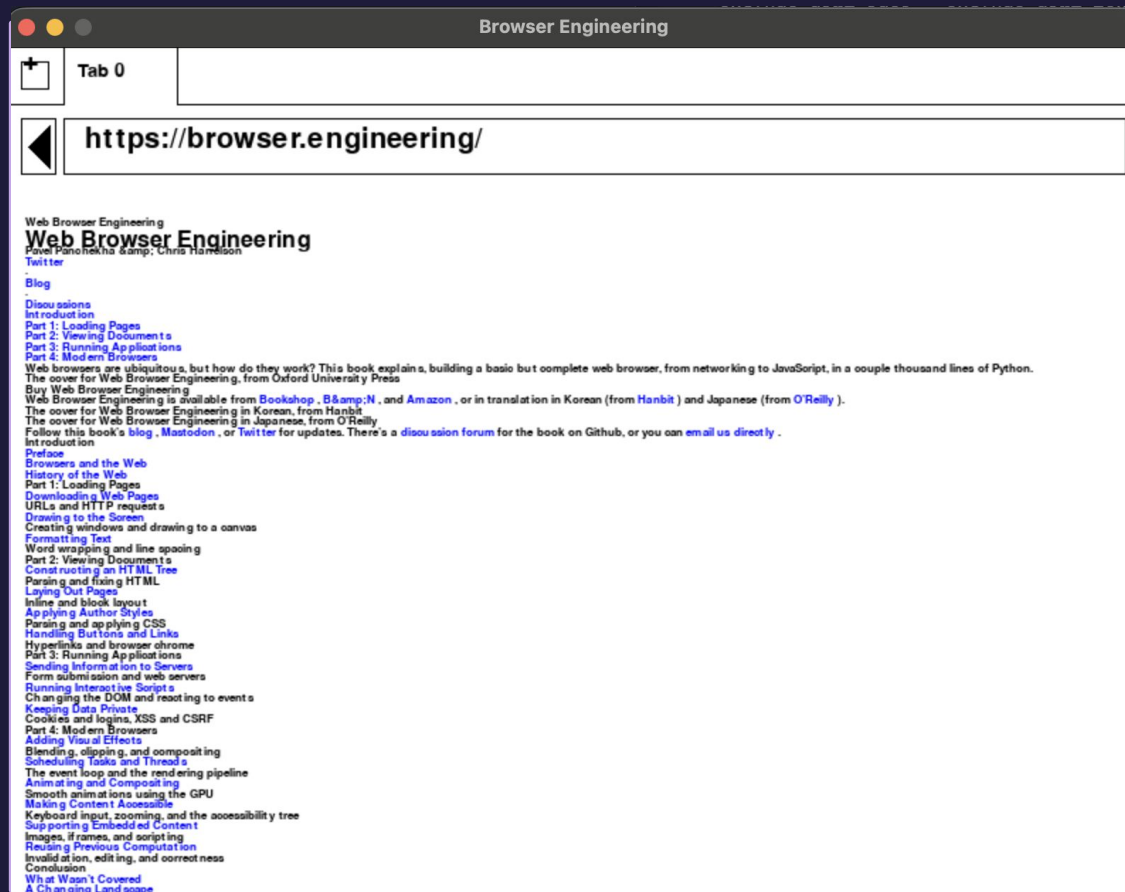
Google

검색 결과...

(x, y, w, h) 계산 → Canvas에 그리기

Ch 4에서 HTML → DOM 트리를 만들고, Ch 2~3에서 그걸 Canvas에 그리는 부분을 손으로 짭니다. Ch 5~6의 Layout·Style은 미도달.

동작 화면



PART 3

책에서 벗어난 과감한 행동

책에서 알려주지 않는 것들이 있어서 여러가지 면에서 색다른 시도를 했습니다



#1 — 책과 다른 추상화로 가기로 하다

Ch1 — `URL.request()` vs `Connection.request()` (의도적 선택)

책의 예제 (어색함)

```
def load(url):  
    body = url.request()  
    show(body)
```

- URL 객체가 소켓을 직접 연다?
- keep-alive 지원하려면 소켓 풀을 어디서 관리해야 하지?
- URL 클래스의 전역 스코프?
인스턴스? ... 둘 다 이상함.

제가 바꾼 버전

```
def load(url):  
    conn = Connection(http_version='1.1')  
    body = conn.request(url=url)  
    show(body)
```

- URL은 '주소'만 가진 값.
- Connection이 TCP handshake와 소켓 풀을 갖고 다님.
- keep-alive는 connection_pool에 소켓을 넣어두는 것으로 자연스럽게.



#2 — 책이 안 알려주는 것 (1) : 이모지의 구현 세부사항

Ch2 연습문제 2.5 – "OpenMoji 기반으로 이모지를 렌더링해주세요" 뒤에 숨어있던 유니코드 이슈들

함정 1 – 참조 잃으면 사라짐

tkinter PhotoImage는

파이썬 변수 참조를 잃는 순간

GC가 메모리에서 날려버림.

→ create_image()가 끝나기도 전에 이미지가
없어짐.

```
emoji_cache = {}  
def load_emoji(path):  
    if path not in emoji_cache:  
        img = tk.PhotoImage(file=path)  
        emoji_cache[path] = img  
    return emoji_cache[path]
```

함정 2 – 이모지는 글자 '여러 개'일 수 있음

🏠 는 하나의 문자가 아니라 3개의 코드포인트 조합:

U+1F468 (사람) +
U+200D (ZWJ, 접합자) +
U+1F4BB (노트북)

→ OpenMoji 파일명 "1F468-200D-1F4BB.png"

→ 하이픈 split + 16진수 파싱 + chr()로 복원해서
매핑.

A #3 — 책이 안 알려주는 것 (2) : 책대로만 하다가 **Linux**에선 실습이 안 됨

Ch3 - tkinter의 `font.measure()`가 런타임마다 너무 다른 문제 (책엔 안 나옴)

뭐가 문제?

macOS: CoreText 네이티브 호출
→ 거의 즉시 반환.

Linux: `measure` 호출마다 X 서버를 경유
→ 레이턴시가 있기 때문에 속도가 체감될 만큼 느림.
→ 렌더링 루프 안에서 호출하면 실습이 '안 됨'.

해결: 인메모리 캐시

```
key = (size, weight, slant, family)
```

```
# 첫 렌더 직전에 한 번만  
prefetch_ascii_widths(font)  
# CJK는 '가' 기준 고정폭으로  
fixed_cjk_width[key] = ...
```

```
# 런타임에는 cache 조회만  
sum(cache[ch] for ch in text)
```

덤: 가변폭 합 ≠ 단어 실폭.
정밀도 ↓ / 속도 ↑ — 실습 목적엔 타협 가능.

실제 브라우저는 이래서 HarfBuzz 같은 크로스 플랫폼 shaping 라이브러리를 쓴다.

덤으로 — 파이썬 생태계 탐험

어차피 파이썬 프로젝트인데, 이참에 안 써본 도구도 다 써보자.



패키징 : poetry → uv

오래 써서 익숙한 *poetry* 두고, 이참에 *uv*로 갈아탐. 빠르고 가벼움.



린터 / 포매터 : black + flake8 → ruff

책에서 시키진 않았지만 깔아둬. PR이 깔끔해지고, Rust 기반이라 압도적으로 빠름.



타입 체커 / Language Server : basedpyright · ty · pyrefly

mypy 말고도 많음. 세 개를 번갈아가며 써봄. 각자 다른 에러를 잡음 — 언어 서버 경험치↑



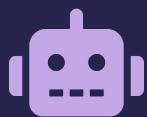
테스트 / 정적분석 : pytest + vulture

*pytest*는 기본. *vulture*로 안 쓰는 죽은 코드를 자동으로 찾아냄.

"이왕 할 거면 욕심대로 실험해본다" — 스터디가 기술 탐험의 핑계가 되어줍니다.

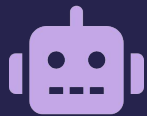
강해지기 위해서 **LLM** 은 가급적이면 안 썼습니다

원칙: 핵심 구현은 손으로, 주변 환경은 LLM 에.



Claude Code

keep-alive 가 실제로 *TCP handshake* 비용을 아끼는지 확인하는 성능 테스트를 맡김. 생각보다 잘 짜줌.



OpenAI Codex

밥 먹으면서 폰으로 *CI* 환경, *FastAPI* 테스트 서버(*리다이렉트*·캐시 응답) 세팅.
"자동사냥" 용으로 적합.



손으로 직접

연습문제 본 풀이, 추상화 설계, 삼절 디버깅.
의도적인 수련이 필요한 부분은 안 맡김.

내가 너무나도 답답해서 책과 다른 방향으로 코드를 짚는데....

아니 근데..

책이 중간중간에 함정 카드를 깔아둡니다.

책이 모듈화도 잘 안해놓고 나이브하게 짜
상요하는걸 내버려둘 순 없어서...

다르게 가면 — 이후 챕터에서 난이도가 누적됩니다.



현상

Ch 1에서 내 방식대로 바꿨더니
Ch 3, Ch 4에서 예제 코드 베낄 때
조금씩 어긋나기 시작.

조용히 누적되는 난이도.



이유

책의 후속 챕터는
앞 챕터 구조를 전제로 씀.

내가 바꾼 구조에 맞추려면
매번 재번역이 필요함.



그래도 권함

그 재번역 과정에서
"내가 왜 이렇게 설계했지"가
계속 단단해집니다.

그게 리터러시입니다.

— 하드코어 모드 스터디가 무서운 진짜 이유. 하지만 그 무서움이 훈련이 됩니다.

PART 4

그리고, 탈주했습니다.

— 가장 먼저 풀던 사람이, 가장 먼저 도망갔습니다.

무슨 일이 있었나요

Ch 1 ~ 4, 순항 중

- 연습문제 전부 풀이 완료
- 해커스펍(블로그)에 기록글 3편 게시
- Connection 추상화 재설계
- 폰트 measure 함수의 성능을 캐싱으로 최적화
- 이모지 / RTL / <sup><sub> 까지 다 밀어넣음

그러던 어느 날...

- 구직 중이었습니다.
- 생각보다 빠르게 합격.
- 온보딩이 들이닥침.
- 스터디는 Ch 5 문턱에서 제 진도가 멈춤.
- 어느 순간, PR도 풀이글도 끊겼습니다.

솔직히 말하면,

끝까지 못 간 게,
아쉽습니다.

CSS, JavaScript, 렌더링 트리, 점진적 리플로우 —
정작 제일 궁금했던 부분은 Ch 5 이후에 나옵니다.
가장 먼저 풀던 사람이, 가장 궁금했던 문턱에서 멈췄어요.

그런데, 남은 게 있더라고요



근육

- 스펙 문서(RFC, WHATWG) 읽는 근육
- "왜 이렇게 설계했지?"를 먼저 묻는 습관
- socket / zlib / tkinter 같은 표준 라이브러리에 대한 이해
- 밑바닥에서부터 구축하는 것에 대한 저항감을 줄였습니다.



기록

- 해커스펍 회고글 3편 (인덱싱되어 남음)
- 다음에 누가 이 책으로 스터디할 때 참고 가능



사람

- 탈주한 뒤에도 스터디는 잘 굴러갔습니다
- "먼저 풀고 간 사람"이 남긴 코드가 하나의 기록물로서 보존이 되었습니다
- 같이 스터디를 했기에 정이 들기도 했습니다.

PART 5

AI가 다 짜주는 시대에, 왜 굳이 밑바닥부터?

— 잠깐, 이 질문 먼저 짚고 가야겠어요.

개인적으로 좋아하는 문구

“만약 재구축할 수 없다면, 당신은 그것을 이해하지 못한 겁니다.”

If you can't rebuild it, you don't understand it. Harsh, but true.

- 좋은 엔지니어는 도구를 씁니다. 위대한 엔지니어는 왜 존재하는지 압니다.
- 좋은 엔지니어는 추상화를 신뢰합니다. 위대한 엔지니어는 그 아래에 무엇이 있는지 압니다.



"일은 항상 망가집니다. 새벽 2시에 추상화가 새면,
한 단계 더 깊이 들어가 본 사람은 당황하지 않고 조사합니다."

"재구축할 수 있는가?"

이 질문이 AI 시대에 오히려 더 날카로워졌습니다.

좋은 엔지니어 (Good)

도구를 사용한다.

튜토리얼을 마친다.

추상화를 신뢰한다.

위대한 엔지니어 (Great)

왜 존재하는지 안다.

그게 어떤 문제를 해결하는지 묻는다.

그 아래에 뭐가 있는지 안다.

망가질 때 조사할 곳을 안다.

차이는 '똑똑함'이 아니라, 망가진 순간에 무력해지기를 거부하느냐입니다.



그리고, 한 가지 더 경계할 것

지금 우리가 쓰는 토큰은,

바겐세일 중입니다.

오늘

LLM에 “해줘”, “만들어줘”를
주저 없이 던집니다.

대부분의 경우엔 이해를 기반하지
않습니다

언젠가

보조금·치킨게임이
끝나는 구간이 옵니다.

토큰당 비용이 지금과
같을 거라는 보장은 없음.

그때의 헤지

LLM 없이도
생각하고 디버깅할 수
있는 '근육'.

그건 만들어본 사람만 가짐.

"리터러시를 기르는 의식적인 훈련이 필요하다" — 가격이 오르기 전에.

처음 제목으로 돌아가면,

모르니까 만들어봤습니다. ■

... 정확히는, '모르니까 일단 만들다가, 다 만들기도 전에 생각하는 방식은 바뀌었습니다'

힘들다가 그만둘 수는 있습니다. 그래도 괜찮다는 말을, 오늘 드리고 싶었어요. — 만든 만큼은 제 것이니까요.

그래서, 원하는 것

추천

<https://codecrafters.io>

<https://codingchallenges.fyi>



매일 쓰는 것 중 하나를 골라 만들어보세요 .

브라우저, 셸, git, DB, HTTP 서버 — 뭐든. 재구축할 수 있는 것 하나가, '새벽 2시에 당황하지 않는 사람'으로 만들어줍니다.



LLM 없이도 돌아가는 '근육'을 남기세요 .

토큰 가격은 영원히 지금 같지 않을 겁니다. 의도적으로 손으로 풀어보는시간을 일정 비율로 확보해두세요.



'하드코어' 모드를 한 번쯤 시도하세요 .

연습문제까지 푸는 스터디는 생각보다 드뭅니다. 쓸데없는 고 퀄로 도전해도 좋습니다. 기억도, 기록도, 다르게 남습니다.



끝까지 못 가도 괜찮습니다 .

중간에 멈춰도 '배운 것'은 도망가지 않아요. 글과 PR로 남겨두면 다음 사람한테도 힌트가 됩니다.

Thank you.

Repo	<code>github.com/kojima-yakshaving/browser-study</code>
Blog	<code>hackers.pub/@kodingwarrior</code> (스터디 기록 1~3)
교재	<code>browser.engineering</code> (Web Browser Engineering)
인용	John Crickett – "If you can't rebuild it, you don't understand it."
Me	Jaeyeol Lee · @kodingwarrior